

SYSTEM DESIGN REPORT

EV Battery Monitoring System Using SocketCAN

Problem Statement 2 — Design and Implement a Distributed System Using SocketCAN

Technology: Python, SocketCAN, Virtual CAN (`vcan0`)

Number of ECUs: 4

1. Problem Description

1.1 Background

Electric Vehicles use battery packs consisting of multiple cells. Continuous monitoring of cell voltage, temperature, current, and battery condition is required for safe operation.

This project implements a software-based EV Battery Monitoring System using SocketCAN. A Linux virtual CAN interface (`vcan0`) is used instead of physical CAN hardware.

The system consists of four independent ECUs:

1. Battery Sensor ECU
2. BMS Controller ECU
3. Power Control ECU
4. Diagnostic ECU

Each ECU has a dedicated responsibility and communicates through the CAN network.

1.2 Objectives

The system is designed to:

- Monitor four battery cell voltages.
- Monitor battery temperature and current.
- Estimate State of Charge (SOC).
- Control charging, discharging, and cooling.
- Communicate using CAN messages.
- Detect abnormal battery conditions.
- Detect missing CAN messages and ECU failures.
- Generate diagnostic information.
- Demonstrate ECU recovery.

2. Functional Architecture

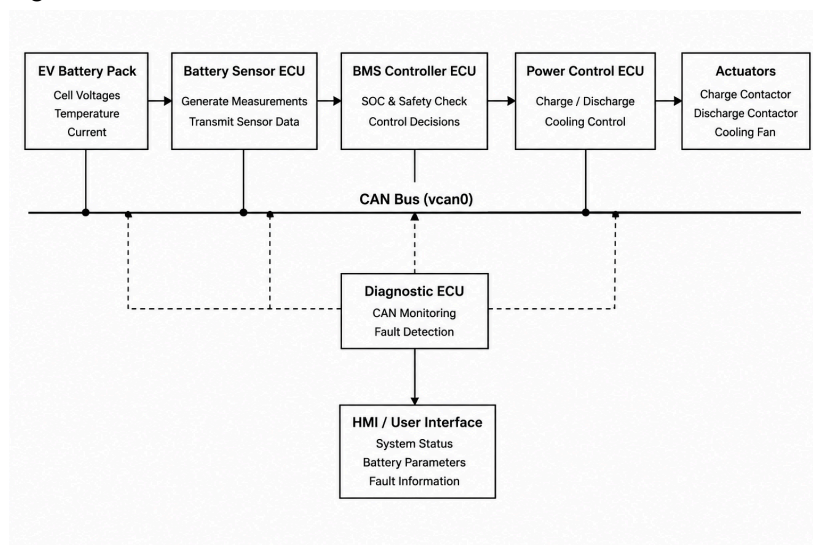
The Battery Sensor ECU generates battery measurements and transmits them through the CAN network.

The BMS Controller receives the measurements, calculates battery parameters, checks safety conditions, and generates control commands.

The Power Control ECU receives these commands and controls simulated charging, discharging, and cooling functions.

The Diagnostic ECU monitors communication and detects faults.

Figure 1 — Functional Architecture



The main inputs are:

- Cell voltages
- Temperature
- Current
- CAN messages
- Simulated fault conditions

The main outputs are:

- Battery status
- SOC
- Charging status
- Discharging status
- Cooling status
- Diagnostic information

3. ECU Architecture

The system contains four independent software ECUs.

3.1 Battery Sensor ECU

The Battery Sensor ECU generates four cell voltages, temperature, current, battery status, and sensor health information.

Messages transmitted:

CAN ID	Message
0x100	Cell Voltage 1–3
0x101	Cell 4 + Temperature + Current
0x102	Battery Pack Status
0x103	Sensor Health

3.2 BMS Controller ECU

The BMS Controller processes sensor data and makes battery safety decisions.

It supports four operating modes:

- IDLE
- CHARGING
- DISCHARGING
- SAFE MODE

It transmits control messages 0x200, 0x201, and 0x202.

3.3 Power Control ECU

The Power Control ECU represents the actuator layer.

It controls:

- Charge contactor

- Discharge contactor
- Cooling fan

It transmits status messages **0x300** and **0x301**.

3.4 Diagnostic ECU

The Diagnostic ECU monitors CAN communication and battery parameters.

It detects:

- Missing messages
- ECU timeouts
- Over-temperature
- Overvoltage
- Undervoltage
- Abnormal current

It transmits diagnostic status using **0x400**.

4. CAN Matrix

The system uses the **vcan0** virtual CAN interface and implements 10 unique CAN messages.

CAN ID	Message	Transmitter	Receiver	Type
0x100	Cell Voltage 1–3	Sensor ECU	Controller, Diagnostic	Periodic
0x101	Cell 4 + Temp + Current	Sensor ECU	Controller, Diagnostic	Periodic
0x102	Battery Pack Status	Sensor ECU	Controller, Diagnostic	Periodic
0x103	Sensor Health	Sensor ECU	Diagnostic	Periodic
0x200	Power Control Command	Controller	Power ECU	Event

0x201	Charging Command	Controller	Power ECU	Event
0x202	Cooling Command	Controller	Power ECU	Event
0x300	Power Control Status	Power ECU	Controller, Diagnostic	Periodic
0x301	Contactor Status	Power ECU	Controller, Diagnostic	Periodic
0x400	Diagnostic Status	Diagnostic ECU	HMI	Periodic

The CAN IDs are grouped logically:

Range	Function
0x100	Sensor information
0x200	Control commands
0x300	Actuator status
0x400	Diagnostics

5. Signal Definitions

5.1 Cell Voltage

Resolution: 0.01 V/bit

Encoding:

$$\text{Encoded Value} = \text{Voltage} \times 100$$

Example:

$$3.72 \text{ V} = 372$$

5.2 Temperature

An offset of 40 is used.

$$\text{Encoded Value} = \text{Temperature} + 40$$

Example:

$$30^{\circ}\text{C} = 70$$

5.3 Current

An offset of 100 is used.

$$\text{Encoded Value} = \text{Current} + 100$$

Example:

$$15 \text{ A} = 115$$

This allows both positive and negative current values.

5.4 SOC

SOC is represented as an integer between 0% and 100% using a simplified voltage-based estimation.

5.5 Battery State

Value	State
1	NORMAL
2	OVERHEAT

3	LOW TEMPERATURE
4	OVERVOLTAGE
5	UNDERVOLTAGE

5.6 Control Signals

The `0x200` message contains:

Byte	Signal
Byte 0	Charge Contactor
Byte 1	Discharge Contactor
Byte 2	Cooling Fan
Byte 3	Operating Mode

6. Design Decisions

SocketCAN was selected because it provides a standard Linux interface for CAN communication.

The `vcan0` virtual CAN interface was selected because it allows complete software-based CAN testing without physical hardware.

Python and the `python-can` library were used because they provide simple CAN communication and allow each ECU to run as an independent application.

The system was divided into four ECUs to represent a distributed automotive architecture and provide clear separation of responsibilities.

A dedicated Diagnostic ECU was implemented to independently monitor network communication and detect failures.

SAFE MODE was implemented for unsafe battery conditions. In this mode, charging and discharging are disabled and cooling is enabled when required.

7. Fault Handling and Validation

The system was tested under normal and abnormal conditions.

Normal operation was verified by running all four ECUs and observing CAN communication.

Missing message detection was tested by stopping the Battery Sensor ECU.

Sensor fault detection was tested by introducing an abnormal temperature value.

Node failure was tested by stopping the Power Control ECU.

Recovery was tested by restarting the failed ECU and verifying that CAN communication resumed.

8. Conclusion

The EV Battery Monitoring System successfully demonstrates a distributed automotive system using SocketCAN.

Four independent ECUs communicate through the `vcn0` virtual CAN network using 10 unique CAN messages. The system performs battery monitoring, SOC estimation, actuator control, diagnostics, fault detection, and recovery.

The project demonstrates the practical concepts of CAN communication, distributed ECU architecture, signal encoding, diagnostics, and system-level fault handling.